

CRIM Уровень преступности

ZN Земли под крупную застройку

INDUS Доля промышленных зон в районе

CHAS Рядом ли река Чарльз

NOX Загрязнение воздуха

RM Среднее кол-во комнат в домах

AGE Доля старых домов

DIS Расстояние до центров занятости

RAD Доступность кольцевых дорог

TAX Налог на имущество

PTRATIO Кол-во учеников на одного учителя

B Доля темнокожих

LSTAT малообеспеченное население

MEDV Медианная цена жилья в \$1000

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
from scipy import stats

sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.dpi"] = 110
```

```
boston = fetch_openml(name="boston", version=1, as_frame=True)
df = boston.frame.copy()
df.columns = [c.upper() for c in df.columns]
df = df.rename(columns={"MEDV": "MEDV"})
df = df.astype(float)

print(df.shape)
df.head()
```

(506, 14)

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98	24.0	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14	21.6	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03	34.7	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94	33.4	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33	36.2	

Далее: [New interactive sheet](#)

✓ Задача 1

Датасет содержит 506 наблюдений без пропущенных значений. MEDV имеет среднее \$22.5 тыс и медиану \$21.2 тыс - правостороннее смещение из-за дорогих районов.

```
# 1.1
print("Размерность:", df.shape)
print("\nТипы данных:")
print(df.dtypes)
```

Размерность: (506, 14)

Типы данных:

CRIM	float64
ZN	float64
INDUS	float64
CHAS	float64
NOX	float64
RM	float64
AGE	float64
DIS	float64
RAD	float64
TAX	float64
PTRATIO	float64
B	float64
LSTAT	float64
MEDV	float64

dtype: object

```
# 1.2
missing = df.isnull().sum()
print("Пропущенные значения по столбцам:")
print(missing[missing > 0] if missing.any() else "Пропущенных значений нет")
```

Пропущенные значения по столбцам:
Пропущенных значений нет

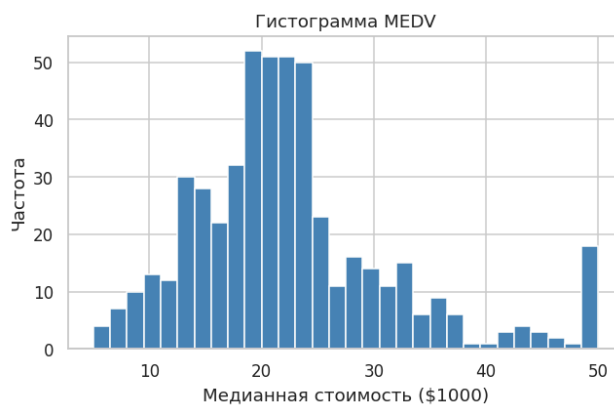
```
# 1.3
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(df["MEDV"], bins=30, color="steelblue", edgecolor="white")
axes[0].set_title("Гистограмма MEDV")
axes[0].set_xlabel("Медианная стоимость ($1000)")
axes[0].set_ylabel("Частота")

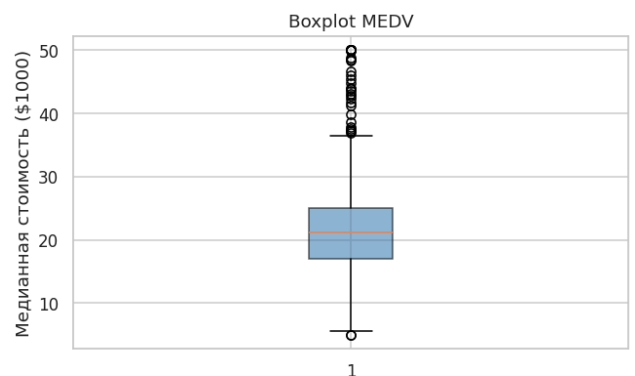
axes[1].boxplot(df["MEDV"], vert=True, patch_artist=True,
                boxprops=dict(facecolor="steelblue", alpha=0.6))
axes[1].set_title("Boxplot MEDV")
axes[1].set_ylabel("Медианная стоимость ($1000)")

plt.tight_layout()
plt.savefig("medv_distribution.png", bbox_inches="tight")
plt.show()

print(f"Среднее: {df['MEDV'].mean():.2f}, Медиана: {df['MEDV'].median():.2f}, "
      f"Std: {df['MEDV'].std():.2f}")
```



Среднее: 22.53, Медиана: 21.20, Std: 9.20



```
# 1.4
correlations = df.corr()["MEDV"].drop("MEDV").sort_values(key=abs, ascending=False)
print("Признаки с наибольшей корреляцией с MEDV:")
print(correlations.head(5).to_string())

plt.figure(figsize=(8, 4))
```

```

correlations.sort_values().plot(kind="barh", color="steelblue", edgecolor="white")
plt.title("Корреляция признаков с MEDV")
plt.xlabel("Коэффициент Пирсона")
plt.tight_layout()
plt.show()

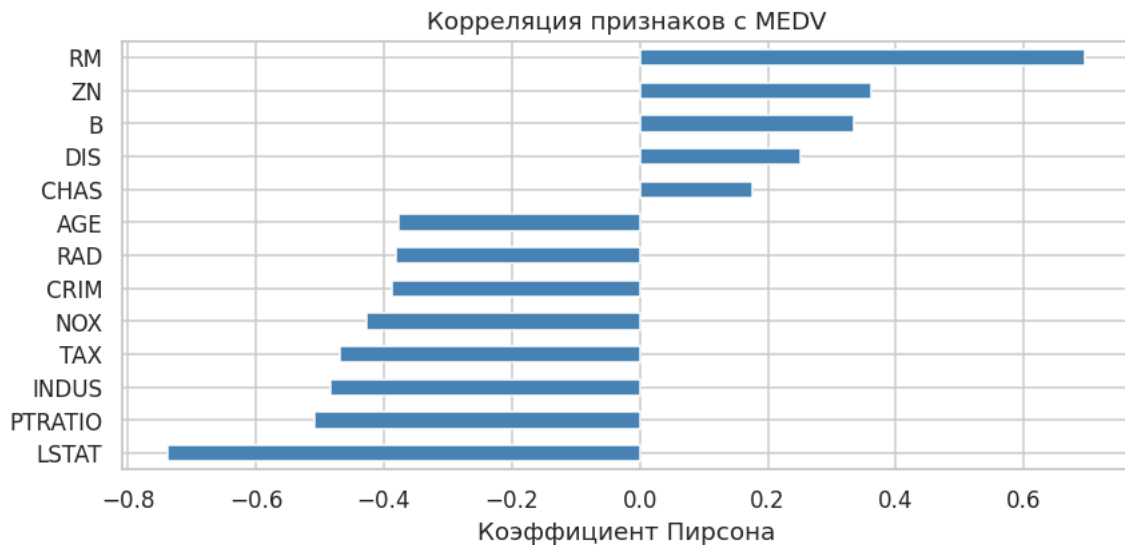
```

Признаки с наибольшей корреляцией с MEDV:

```

LSTAT    -0.737663
RM        0.695360
PTRATIO  -0.507787
INDUS    -0.483725
TAX      -0.468536

```



Задание 2

Нормализованы с помощью StandardScaler и теперь все данные имеют одинаковый масштаб. Создан признак is_high_value: 84 района имеют цену выше \$30k. После фильтрации удалено 91 наблюдение.

```

# 2.1
feature_cols = [c for c in df.columns if c != "MEDV"]
scaler = StandardScaler()

df_scaled = df.copy()
df_scaled[feature_cols] = scaler.fit_transform(df[feature_cols])

print("После нормализации (первые 3 строки):")
df_scaled[feature_cols].head(3)

```

После нормализации (первые 3 строки):

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
0	-0.419782	0.284830	-1.287909	-0.272599	-0.144217	0.413672	-0.120013	0.140214	-0.982843	-0.666608	-1.459000	0.441052
1	-0.417339	-0.487722	-0.593381	-0.272599	-0.740262	0.194274	0.367166	0.557160	-0.867883	-0.987329	-0.303094	0.441052
2	-0.417342	-0.487722	-0.593381	-0.272599	-0.740262	1.282714	-0.265812	0.557160	-0.867883	-0.987329	-0.303094	0.396427

```

# 2.2
df_scaled["is_high_value"] = (df["MEDV"] > 30).astype(int)
print("Распределение is_high_value:")
print(df_scaled["is_high_value"].value_counts())

```

```

Распределение is_high_value:
is_high_value
0      422
1       84
Name: count, dtype: int64

```

```

# 2.3
z_scores = np.abs(stats.zscore(df_scaled[feature_cols + ["MEDV"]]))
mask = (z_scores < 3).all(axis=1)

df_clean = df_scaled[mask].reset_index(drop=True)
df_orig_clean = df[mask].reset_index(drop=True)

print(f"Строк до удаления выбросов: {len(df_scaled)}")

```

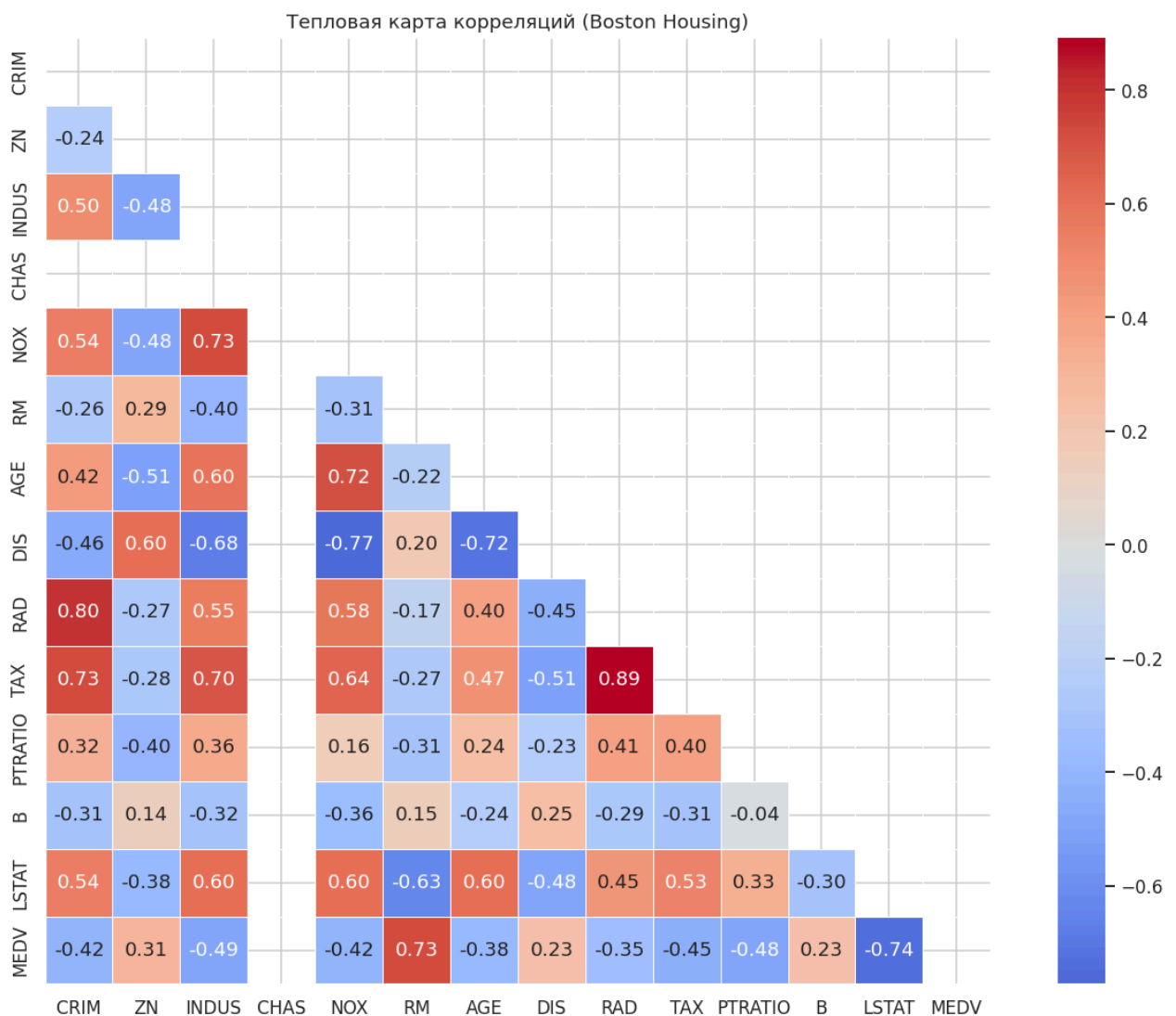
```
print(f"Строк после удаления выбросов: {len(df_clean)}")
print(f"Удалено: {len(df_scaled) - len(df_clean)} строк")
```

Строк до удаления выбросов: 506
Строк после удаления выбросов: 415
Удалено: 91 строк

Задание 3

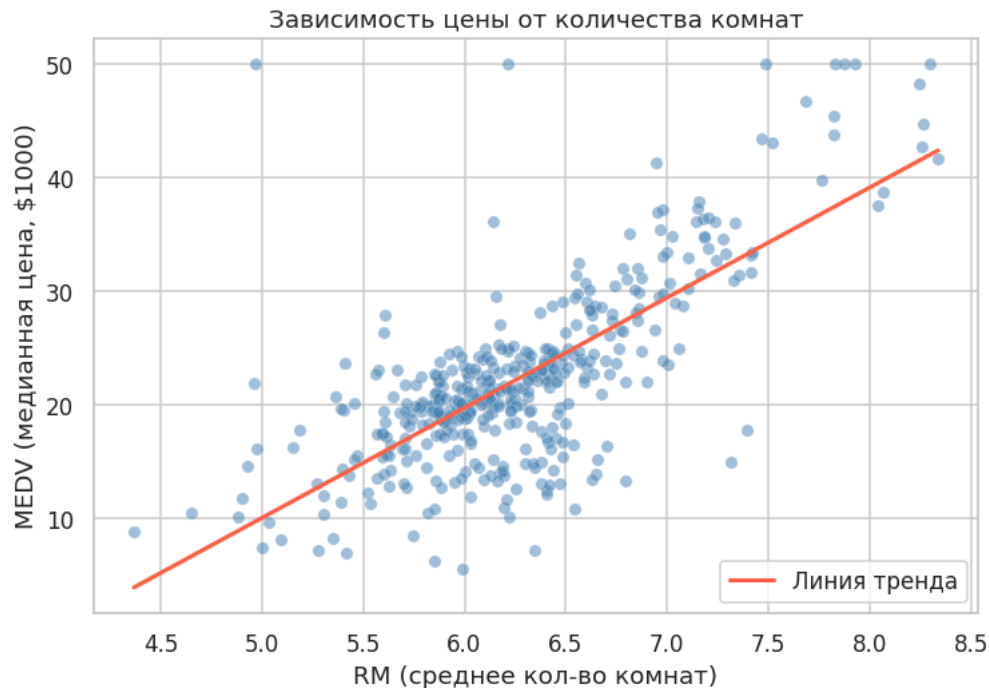
По тепловой карте нашлась сильная корреляция между RAD и TAX, между RAD и CRIM и между NOX и DIS.

```
# 3.1
plt.figure(figsize=(12, 9))
corr_matrix = df_orig_clean.drop(columns=["is_high_value"], errors="ignore").corr()
mask_upper = np.triu(np.ones_like(corr_matrix, dtype=bool))
sns.heatmap(corr_matrix, mask=mask_upper, annot=True, fmt=".2f",
            cmap="coolwarm", center=0, linewidths=0.5, square=True)
plt.title("Тепловая карта корреляций (Boston Housing)")
plt.tight_layout()
plt.savefig("heatmap.png", bbox_inches="tight")
plt.show()
```

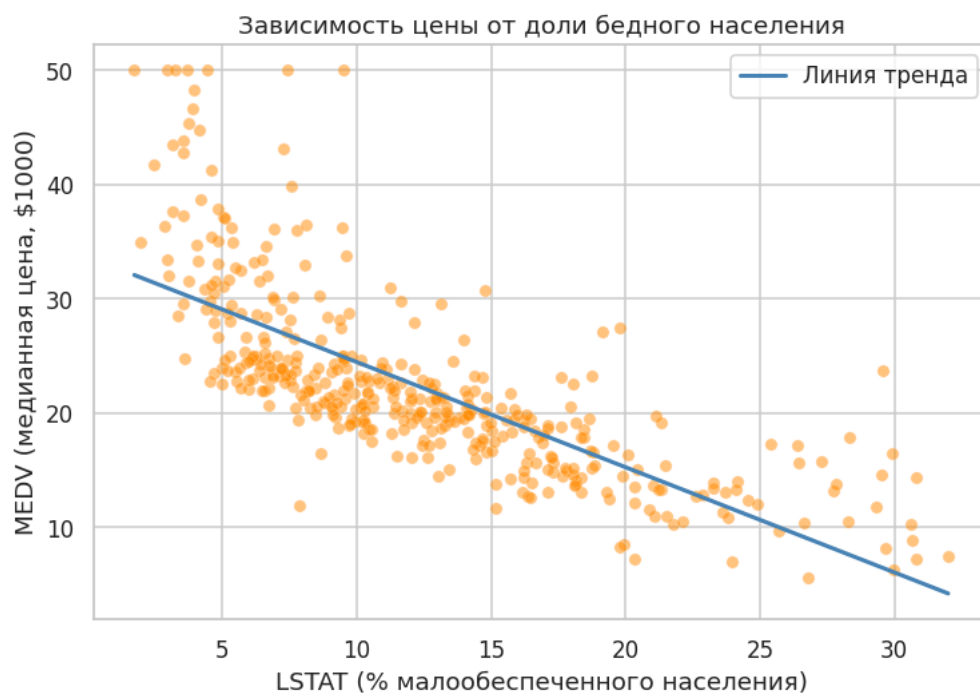


```
# 3.2
plt.figure(figsize=(7, 5))
plt.scatter(df_orig_clean["RM"], df_orig_clean["MEDV"],
            alpha=0.5, color="steelblue", edgecolors="white", linewidths=0.3)
m, b = np.polyfit(df_orig_clean["RM"], df_orig_clean["MEDV"], 1)
```

```
x_line = np.linspace(df_orig_clean["RM"].min(), df_orig_clean["RM"].max(), 100)
plt.plot(x_line, m * x_line + b, color="tomato", linewidth=2, label="Линия тренда")
plt.xlabel("RM (среднее кол-во комнат)")
plt.ylabel("MEDV (медианная цена, $1000)")
plt.title("Зависимость цены от количества комнат")
plt.legend()
plt.tight_layout()
plt.show()
```



```
# 3.3
plt.figure(figsize=(7, 5))
plt.scatter(df_orig_clean["LSTAT"], df_orig_clean["MEDV"],
            alpha=0.5, color="darkorange", edgecolors="white", linewidths=0.3)
m2, b2 = np.polyfit(df_orig_clean["LSTAT"], df_orig_clean["MEDV"], 1)
x_line2 = np.linspace(df_orig_clean["LSTAT"].min(), df_orig_clean["LSTAT"].max(), 100)
plt.plot(x_line2, m2 * x_line2 + b2, color="steelblue", linewidth=2, label="Линия тренда")
plt.xlabel("LSTAT (% малообеспеченного населения)")
plt.ylabel("MEDV (медианная цена, $1000)")
plt.title("Зависимость цены от доли бедного населения")
plt.legend()
plt.tight_layout()
plt.show()
```



Задание 4

Дисперсия показала 74%, что неплохо для трёх признаков. В цену наибольший вклад даёт количество комнат. $RMSE > MSE \Rightarrow$ есть большие ошибки

```
# 4.1
features = ["RM", "LSTAT", "PTRATIO"]
X = df_orig_clean[features].values
y = df_orig_clean["MEDV"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
```

```
# 4.2
coef_df = pd.DataFrame({
    "Признак": features,
    "Коэффициент": lr.coef_
})
print("Intercept (свободный член):", round(lr.intercept_, 4))
print(coef_df.to_string(index=False))
```

```
Intercept (свободный член): 11.1544
Признак  Коэффициент
      RM      5.288025
    LSTAT -0.525218
    PTRATIO -0.836686
```

```
# 4.3
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"R2 = {r2:.4f}")
print(f"MAE = {mae:.4f}")
print(f"RMSE = {rmse:.4f}")
```

```
R2 = 0.7368
MAE = 2.6858
RMSE = 3.5337
```

Задание 5

Делаем вот такую формулу $\beta = (X^T X)^{-1} X^T y$; Что по сути и делает sklearn. Коэффициенты совпали, так что всё правильно

```
# 5.1
X_train_b = np.hstack([np.ones((X_train.shape[0], 1)), X_train])
X_test_b = np.hstack([np.ones((X_test.shape[0], 1)), X_test])

# β
beta = np.linalg.inv(X_train_b.T @ X_train_b) @ X_train_b.T @ y_train

print("Коэффициенты вручную (NumPy):")
print(f" Intercept : {beta[0]:.4f}")
for name, coef in zip(features, beta[1:]):
    print(f" {name:8s}: {coef:.4f}")

print("\nКоэффициенты sklearn:")
print(f" Intercept : {lr.intercept_:.4f}")
for name, coef in zip(features, lr.coef_):
    print(f" {name:8s}: {coef:.4f}")
```

```
Коэффициенты вручную (NumPy):
Intercept : 11.1544
      RM   : 5.2880
    LSTAT : -0.5252
    PTRATIO : -0.8367
```

```
Коэффициенты sklearn:
Intercept : 11.1544
      RM   : 5.2880
    LSTAT : -0.5252
    PTRATIO : -0.8367
```

```
# 5.2
y_pred_manual = X_test_b @ beta
r2_m = r2_score(y_test, y_pred_manual)
mae_m = mean_absolute_error(y_test, y_pred_manual)
rmse_m = np.sqrt(mean_squared_error(y_test, y_pred_manual))

print(f"NumPy - R²: {r2_m:.4f}, MAE: {mae_m:.4f}, RMSE: {rmse_m:.4f}")
print(f"sklearn - R²: {r2:.4f}, MAE: {mae:.4f}, RMSE: {rmse:.4f}")
print("\nКоэффициенты совпадают ✓" if np.allclose(beta[1:], lr.coef_, atol=1e-6) else "Расхождение!")

NumPy - R²: 0.7368, MAE: 2.6858, RMSE: 3.5337
sklearn - R²: 0.7368, MAE: 2.6858, RMSE: 3.5337

Коэффициенты совпадают ✓
```

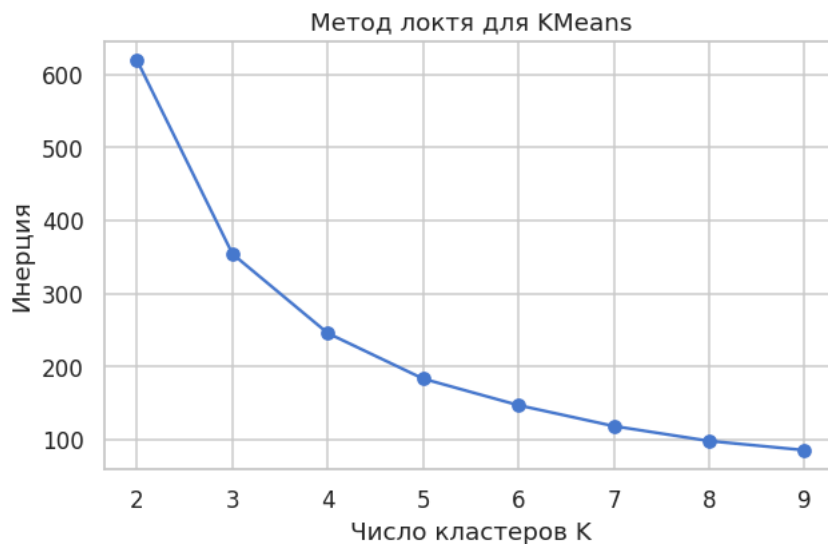
Задание 6

Кластер 0 - загрязнённые центральные районы со средней ценой \$15.9тыщ. Кластер 1 - чистые удалённые пригороды, \$24.5тыщ. Кластер 2 - средние районы, \$23.6тыщ.

```
# 6.1
cluster_features = ["CRIM", "NOX", "DIS"]
X_cluster = df_orig_clean[cluster_features].copy()
scaler_cl = StandardScaler()
X_cluster_scaled = scaler_cl.fit_transform(X_cluster)

inertias = []
K_range = range(2, 10)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_cluster_scaled)
    inertias.append(km.inertia_)

plt.figure(figsize=(6, 4))
plt.plot(K_range, inertias, "bo-")
plt.xlabel("Число кластеров K")
plt.ylabel("Инерция")
plt.title("Метод локтя для KMeans")
plt.tight_layout()
plt.show()
```



```
# 6.2
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df_orig_clean["Cluster"] = kmeans.fit_predict(X_cluster_scaled)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
colors = ["steelblue", "darkorange", "seagreen"]

for cl in sorted(df_orig_clean["Cluster"].unique()):
    subset = df_orig_clean[df_orig_clean["Cluster"] == cl]
    axes[0].scatter(subset["CRIM"], subset["NOX"], label=f"Кластер {cl}",
                    alpha=0.6, color=colors[cl], edgecolors="white", linewidths=0.3)
    axes[1].scatter(subset["DIS"], subset["NOX"], label=f"Кластер {cl}",
                    alpha=0.6, color=colors[cl], edgecolors="white", linewidths=0.3)
```

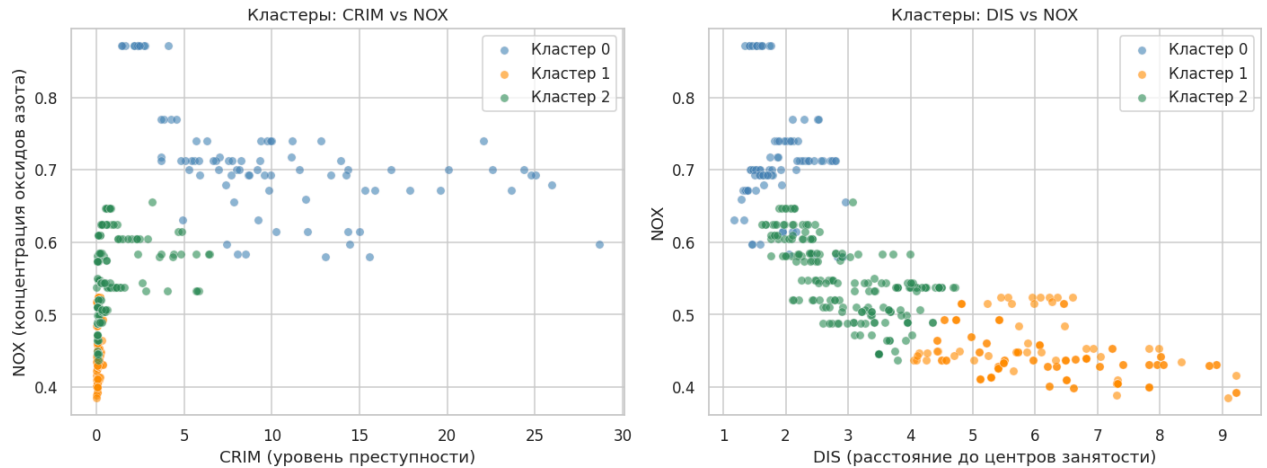
```

axes[0].set_xlabel("CRIM (уровень преступности)")
axes[0].set_ylabel("NOX (концентрация оксидов азота)")
axes[0].set_title("Кластеры: CRIM vs NOX")
axes[0].legend()

axes[1].set_xlabel("DIS (расстояние до центров занятости)")
axes[1].set_ylabel("NOX")
axes[1].set_title("Кластеры: DIS vs NOX")
axes[1].legend()

plt.tight_layout()
plt.show()

```



```

# 6.3
cluster_stats = df_orig_clean.groupby("Cluster")["MEDV"].agg(["mean", "median", "count"])
cluster_stats.columns = ["Средняя цена", "Медианная цена", "Кол-во районов"]
print(cluster_stats.round(2))

plt.figure(figsize=(6, 4))
cluster_stats["Средняя цена"].plot(kind="bar", color=colors, edgecolor="white", rot=0)
plt.title("Средняя цена жилья по кластерам")
plt.xlabel("Кластер")
plt.ylabel("MEDV ($1000)")
plt.tight_layout()
plt.show()

```


Cluster	Средняя цена	Медианная цена	Кол-во районов
0	15.90	14.60	85
1	24.50	23.15	142
2	23.65	21.75	188

